

11246062 邱偉豪、11246064 張庭語、11246084 鍾百陶、11246085 張家瑜

資管三乙

專題學習進度檢核報告

KERAS大神歸位:深度學習全面進化!用 PYTHON 實作CNN、
RNN、GRU、LSTM、GAN、VAE、TRANSFORMER

指導教授：蒯思齊

2026/01/27
WEEK2

4-2 分類數位新聞專欄： 多類別分類範例

主講：張庭語

相關檔案

- [Google Colab](#)
- [Google Docs](#)

- 1 路透社 (Reuters) 資料集
- 2 準備資料
- 3 建立神經模型
- 4 驗證模型表現
- 5 對新資料進行預測
- 6 處理標籤與損失的另一種方式
- 7 擁有足夠大型中間層的重要性

路透社資料集

前言

上一章節，將向量輸入分類是兩個不同的類別，但如果遇到兩個以上不同的類別時應該麼辦？

4-2要構建一個神經網路。將路透社的數位新聞專欄分成46個完全不同的主題，在此將探討「多類別分類」的問題。

路透社資料集也內建在Keras中

路透社資料集中的新聞總共有46個不同的主題。且IMDB、MNIST一樣，路透社資料集也內建在Keras中：

```
[1] 17 秒
✓ # 從tensorflow.keras.datasets模組中導入reuters資料集
  from tensorflow.keras.datasets import reuters

  # 載入reuters資料集，並將其分為訓練集和測試集
  # num_words=10000 表示只保留訓練資料中最常出現的10000個單詞
  (train_data, train_labels), (test_data, test_labels) = reuters.load_data(num_words=10000)

  ... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/reuters.npz
  2110848/2110848 ----- 1s lus/step
```

接著，我們檢視一下總共會得到多少個訓練樣本以及測試樣本：

```
[2] 0 秒
✓ # 計算 train_data 中元素的數量，即訓練集中的新聞文章數量，並將結果輸出。
  len(train_data)

  8982

[3] 0 秒
✓ # 計算 test_data 中元素的數量，即測試集中的新聞文章數量，並將結果輸出。
  len(test_data)

  ... 2246
```

總共有8982個訓練樣本、2246個訓練樣本

路透社資料集

將編號解碼回單字

[4]
✓ 0 秒

```
# 取得路透社資料集中的單字索引對應字典，鍵是單字，值是整數索引。  
word_index = reuters.get_word_index()  
  
# 將單字索引字典反轉，變成鍵是整數索引，值是單字的字典，方便將數字轉換回單字。  
reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])  
  
# 將訓練資料中的第一篇新聞文章（train_data[0]）的數字序列解碼回可讀的單字串列  
# 減去3是因為索引0、1、2是特殊保留字元，例如填充（padding）、序列開始（start of sequence）和未知單字（unknown）。如果索引減3後在反轉字典中找不到，則預設為'?'。  
decoded_newswire = ' '.join([reverse_word_index.get(i - 3, '?') for i in train_data[0]])  
  
# 輸出解碼後的新聞文章  
print(decoded_newswire)
```

▼
? ? ? said as a result of its december acquisition of space co it expects earnings per share in 1987 of 1 15 to 1 30 dlrs per share up from 70 cts in 1986 the compa

準備資料

延續使用先前資料

對資料進行向量化處理：

```
# 匯入 NumPy 函式庫，並將其別名設定為 np，NumPy 用於高效地處理陣列和數值運算。
import numpy as np

# 定義一個名為 vectorize_sequences 的函式，用於將序列資料向量化。
# 函式接受兩個參數：sequences（資料序列，如電影評論的數字列表）和 dimension（向量的維度，預設為 10000）。
def vectorize_sequences(sequences, dimension = 10000):

    # 創建一個全為零的 NumPy 陣列，形狀為（序列數量，維度）。
    # 這個陣列將用於儲存向量化後的資料，每一列代表一個評論，每個元素代表一個單字是否出現。
    results = np.zeros((len(sequences), dimension))

    # 迭代處理每一個輸入序列及其索引。
    # i 是序列在 `sequences` 中的索引，sequence 是該評論的數字列表。
    for i, sequence in enumerate(sequences):

        # 將 `results` 陣列中對應序列位置和單字索引的位置設定為 1。
        # 這表示在當前評論（第 i 列）中，索引為 `sequence` 中的單字是存在的。
        # 這種轉換稱為「詞袋模型（Bag-of-Words）」或「one-hot 編碼」。
        results[i, sequence] = 1

    # 函式返回向量化後的結果陣列。
    return results

# 呼叫 vectorize_sequences 函式，將訓練資料 train_data 轉換為向量化表示。
# x_train 將是一個二維 NumPy 陣列，每一行是一個訓練評論的向量表示。
x_train = vectorize_sequences(train_data)

# 呼叫 vectorize_sequences 函式，將測試資料 test_data 轉換為向量化表示。
# x_test 將是一個二維 NumPy 陣列，每一行是一個測試評論的向量表示。
x_test = vectorize_sequences(test_data)
```


準備資料

使用one-hot編碼

函式會將標籤向量的每個元素設為0，只有指定的索引位置是1

```
[11]:
✓ 0
秒

def to_one_hot(labels, dimension = 46):

    # 先將所有元素設為0，創建一個指定形狀（標籤數量 x 維度）的全零NumPy陣列。
    # 這個陣列將用於儲存 one-hot 編碼後的結果。
    results = np.zeros((len(labels), dimension))
    # 迭代處理每一個輸入標籤及其索引。
    # i 是標籤在 `labels` 中的索引，label 是該標籤的數值。
    for i, label in enumerate(labels):
        # 將 `results` 陣列中對應的標籤位置設定為 1.0。
        # 這表示在第 `i` 個樣本中，其類別是 `label` 所代表的索引。
        results[i, label] = 1.
    # 函式返回 one-hot 編碼後的結果陣列。
    return results

# 呼叫 to_one_hot 函式，將訓練集標籤 train_labels 轉換為 one-hot 編碼的向量化表示。
# y_train 將是一個二維 NumPy 陣列，每一行是一個訓練標籤的 one-hot 表示。
y_train = to_one_hot(train_labels)
# 呼叫 to_one_hot 函式，將測試集標籤 test_labels 轉換為 one-hot 編碼的向量化表示。
# y_test 將是一個二維 NumPy 陣列，每一行是一個測試標籤的 one-hot 表示。
y_test = to_one_hot(test_labels)
```

進行向量化的兩種方式

1. 將標籤串列轉換為整數張量。
2. one-hot編碼，將資料編碼成0跟1的數值，
 - a. 被廣泛應用在「分類資料格式(又稱：分類編碼)」

建立神經模型

更大的層

前一章節我們使用16維的中間層，但是16維空間太有限，使模型無法學會區分46個不同的類別...

為了解決這個問題，我們需要更大的層(單元數更多)，下方試著使用64維的層：

```
[53] 0  
✓ 秒 #from tensorflow import keras  
#from tensorflow.keras import layers  
  
model = keras.Sequential([  
    layers.Dense(64, activation='relu'),  
    layers.Dense(64, activation='relu'),  
    layers.Dense(46, activation='softmax')  
])
```

定義模型

說明

使用一個大小是46單元的Dense(密集)層結束神經網路，這表示：

1. 將會輸出一個46維的向量。向量中的每一項目都會對應到一個不同的輸出類別。
2. 最後一層使用softmax激活函數，針對46個不同的輸出類別輸入機率分布。

損失函數

使用損失函數categorical_crossentropy，它可以測量出兩個機率分布的距離，就是神經網路輸出的分布與標籤真實分類之間的距離。

```
[56] 0  
✓ 秒 model.compile(optimizer='rmsprop',  
                  loss='categorical_crossentropy',  
                  metrics=['accuracy'])
```


驗證模型表現

抽出1,000個樣本，作為驗證資料集

```
[60]  
✓ 0 秒  
x_val = x_train[:1000]  
  
partial_x_train = x_train[1000:]  
  
y_val = y_train[:1000]  
  
partial_y_train = y_train[1000:]
```

訓練20個週期

```
[59]  
✓ 24 秒  
history = model.fit(partial_x_train,  
                    partial_y_train,  
                    epochs = 20,  
                    batch_size = 512,  
                    validation_data = (x_val, y_val))  
  
Epoch 1/20  
16/16 — 3s 107ms/step - accuracy: 0.3374 - loss: 3.3147 - val_accuracy: 0.6030 - val_loss: 1.9800  
Epoch 2/20  
16/16 — 1s 69ms/step - accuracy: 0.6506 - loss: 1.7361 - val_accuracy: 0.6840 - val_loss: 1.4243  
Epoch 3/20  
16/16 — 1s 48ms/step - accuracy: 0.7254 - loss: 1.3900 - val_accuracy: 0.7270 - val_loss: 1.3186
```

繪製其損失以及準確性曲線圖

1. 使用matplotlib繪製圖表
2. 訓練和驗證損失的圖
3. 訓練和驗證準確度的圖

由於神經網路的隨機初始化不同，因此在執行上結果可能會與書中不同。

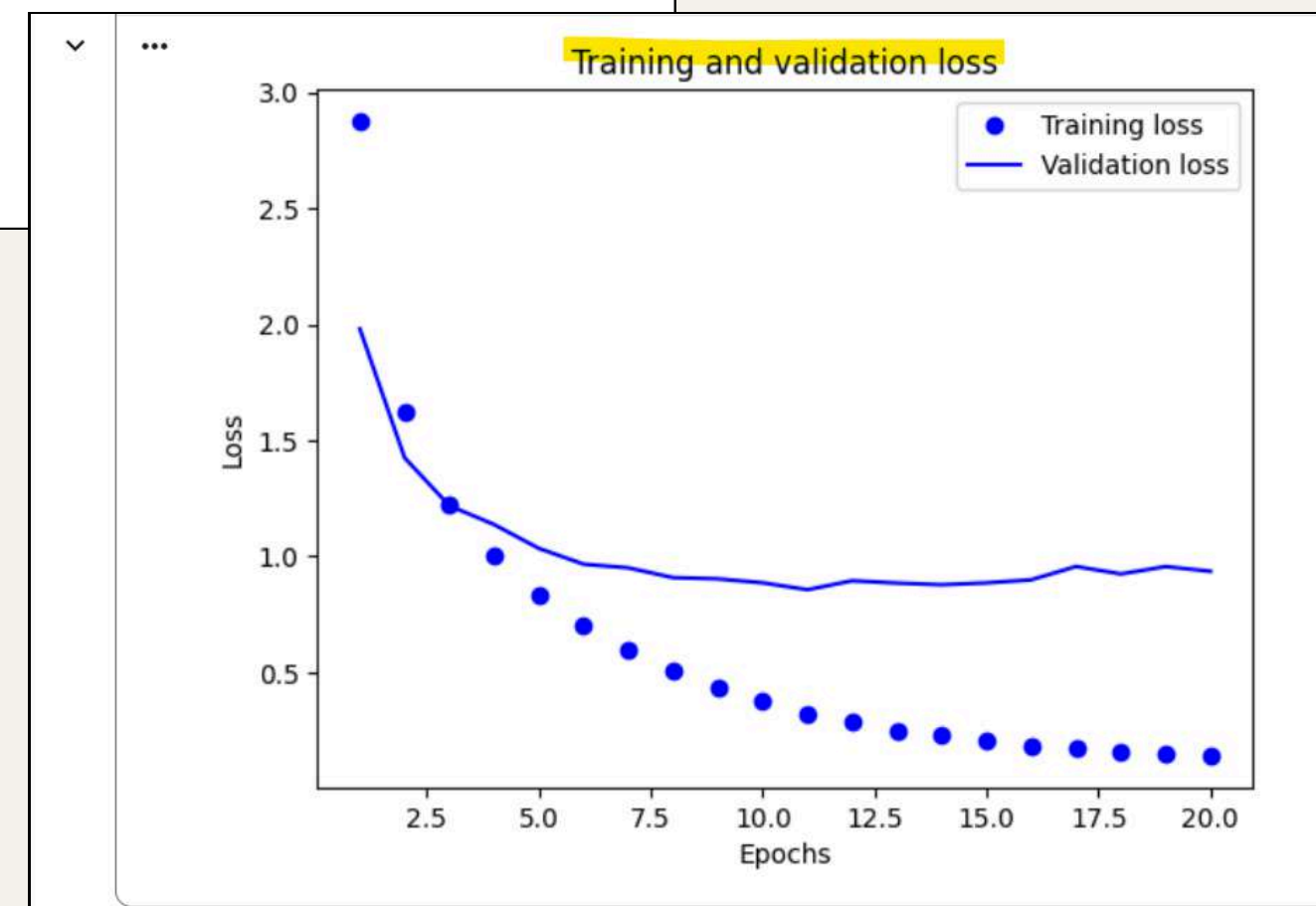
驗證模型表現

繪製訓練和驗證損失圖

```
[67] 0 秒  
✓  
# 繪製訓練和驗證損失圖  
import matplotlib.pyplot as plt  
  
loss = history.history['loss']  
val_loss = history.history['val_loss']  
  
epochs = range(1, len(loss) + 1)  
  
plt.plot(epochs, loss, 'bo', label='Training loss')  
plt.plot(epochs, val_loss, 'b', label='Validation loss')  
plt.title('Training and validation loss')  
plt.xlabel('Epochs')  
plt.ylabel('Loss')  
plt.legend()  
  
plt.show()
```

訓練損失與驗證損失趨勢說明：

訓練損失在訓練過程中持續下降，代表模型在訓練資料上的表現越來越好；然而驗證損失僅在初期下降，約在第 10 ~ 11 個 epoch 之後開始出現波動並略為上升，顯示模型逐漸發生過度擬合，對新資料的泛化能力下降。



驗證模型表現

繪製訓練和驗證準確度圖

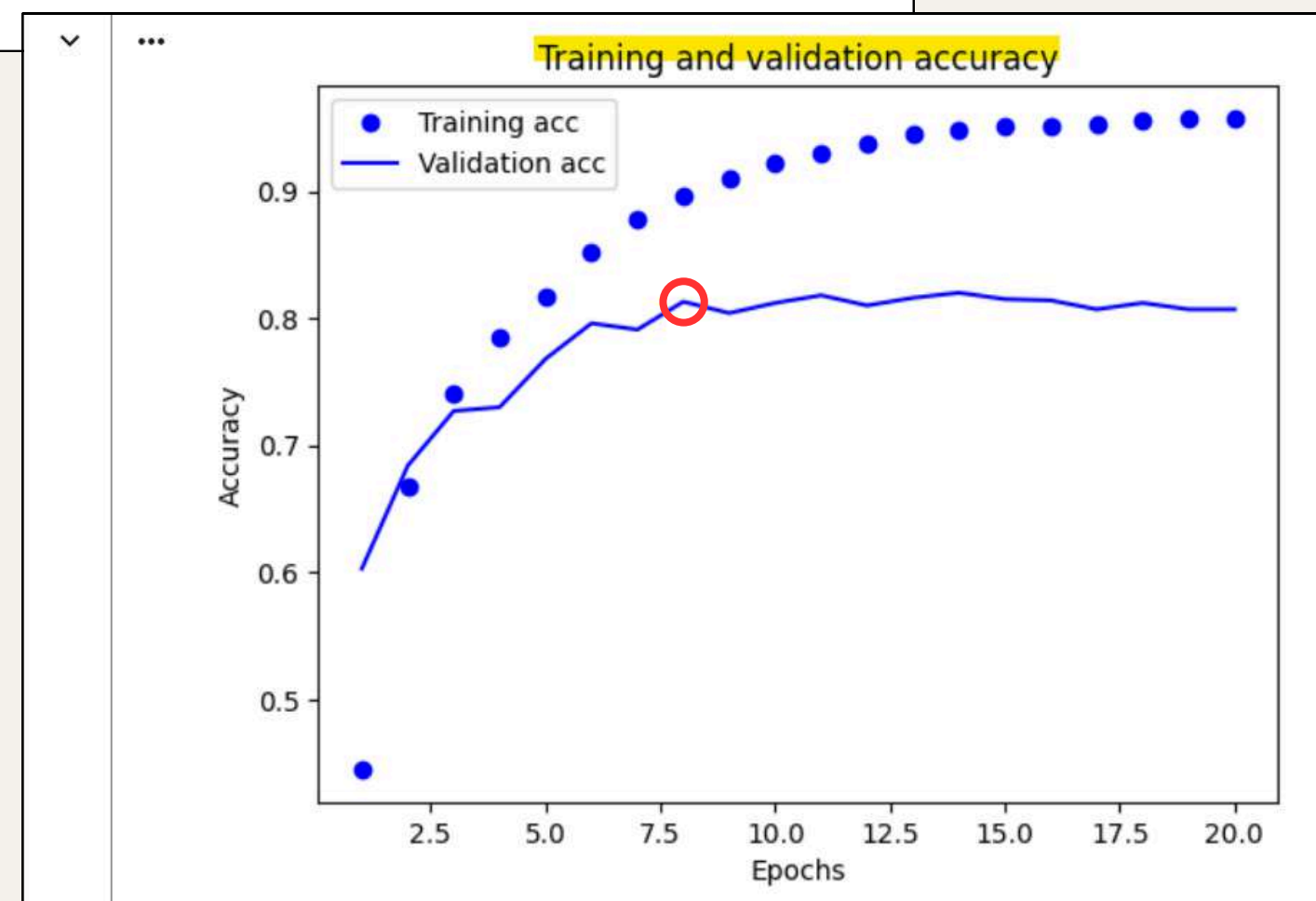
```
[68]
✓ 0 秒
# 繪製訓練和驗證準確性圖

# 先清除畫面
plt.clf()
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```

訓練準確度在整個訓練過程中持續且明顯提升，顯示模型不斷加強對訓練資料的學習；相對地，驗證準確度約在第 10~11 個 epoch 達到最高值（約 0.818）後便趨於停滯，甚至在後期略有下降（如第 17 與第 19 個 epoch 約為 0.806），即使訓練準確度仍持續上升，仍顯示模型開始出現過度擬合現象。



對新資料進行預測

使用predict方法

使用predict方法，取得46個主題的機率分布。

首先，為測試資料做主題預測：

```
[87]  
✓ 0  
秒  
# 首先，為測試資料做主題預測  
predictions = model.predict(x_test)  
  
# predictions陣列中的每個項目都是長度等於46的向量  
predictions[0].shape  
  
# 該向量中的元素總和等於1（機率總和等於1）  
np.sum(predictions[0])  
  
# 擁有最大數值的項目就是模型預測該樣本所屬的類別（也就是最高機率的類別）  
np.argmax(predictions[0])  
  
... 71/71 ————— 0s 3ms/step  
np.int64(3)
```

說明

- `model.predict(x_test)` 會讓模型對測試集中的所有新聞文章進行預測，並輸出每篇文章的類別機率分佈。
- 其中，`predictions[0]` 為測試集中第一篇新聞的預測結果，是一個包含 46 個數值的向量，每個數值代表該新聞屬於對應類別的機率，且這些機率加總約為 1。
- 透過 `np.argmax(predictions[0])` 可找出機率最高的類別索引，例如輸出為 3，表示模型判定該新聞最可能屬於第 3 類。

處理標籤與損失的 另一種方式

使用另一種編碼方式 將它們轉換為整數張量

```
[88]
✓ 0 秒
y_train = np.array(train_labels)
y_test  = np.array(test_labels)
```

說明

這種方式會改變損失函數的選擇：

- 損失函數categorical_crossentropy，僅適用於one-hot編碼部分。
- 若要使用整數標籤，就應該使用損失函數sparse_categorical_crossentropy。

```
[91]
✓ 0 秒
model.compile(optimizer='rmsprop',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])
```

兩個損失函數在數學上是相同的，只是輸入參數的格式不同：

損失函數	categorical_crossentropy	sparse_categorical_crossentropy
輸入參數的格式	[1,0,0]	1
	[0,1,0] 分類標籤	2 整數標籤
	[0,0,1]	3

擁有足夠大型中間層的重要性

概念說明

4-2 章節的最終輸出是 46 維，因此，應該要避免使用少於 46 個單元的中間層。如果使用了少於 46 個單元的中間層，將發生怎樣的情況呢？

```
[123] 33 秒
from tensorflow import keras # 確保 keras 模組已匯入
from tensorflow.keras import layers # 確保 layers 模組已匯入

model = keras.Sequential() # 將 models.Sequential() 修改為 keras.Sequential() 以修正 NameError
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))
# 為了測試，故意將中間層改為4個單元
model.add(layers.Dense(4, activation='relu'))
model.add(layers.Dense(46, activation='softmax'))

model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(partial_x_train,
          partial_y_train,
          epochs=20,
          batch_size=128,
          validation_data=(x_val, y_val))
```

63/63	2s	28ms/step	- accuracy: 0.8638	- loss: 0.5426	- val_accuracy: 0.7130	- val_loss: 1.4664
Epoch 18/20						
63/63	1s	18ms/step	- accuracy: 0.8742	- loss: 0.4875	- val_accuracy: 0.7260	- val_loss: 1.5152
Epoch 19/20						
63/63	1s	17ms/step	- accuracy: 0.8868	- loss: 0.4660	- val_accuracy: 0.7190	- val_loss: 1.5672
Epoch 20/20						
63/63	1s	17ms/step	- accuracy: 0.8865	- loss: 0.4510	- val_accuracy: 0.7110	- val_loss: 1.6089

<keras.src.callbacks.history.History at 0x7e36ac09daf0>

最終結果

模型的驗證準確度達到 71%，較先前的準確度相比，降低了 8%。

主因是，試圖壓縮大量資訊到一個低維度的中間層表示空間，神經網路可以將大部分的重要資訊塞進 4 維表示法中，但並非全部，因此難免會犧牲部分資訊，進而造成驗證準確度下降。

CH8 電腦視覺的 深度學習簡介

主講：張庭語

1 前導簡介

2 卷積神經網路

3 以少量資料集從頭訓練一個
卷積神經網路

4 使用預先訓練好的模型

相關檔案

- [8-1 Google Colab](#)
- [8-2 與 8-3Google Colab](#)
- [Google Docs](#)

前導簡介

深度學習最早、最成功的應用領域是...?

電腦視覺是深度學習最早、也最成功的應用領域之一，廣泛應用於圖片搜尋、影片平台、OCR、AI 濾鏡、**無人駕駛**與醫療等



只有兩扇鷗翼門、沒方向盤或油門踏板的自駕計程車Cybercab



自駕廂型車Robovan的概念車，一次可載運20人

資料來源：[商業週刊](#)

電腦視覺的關鍵模型

卷積神經網路（CNN），專門用於影像相關任務，尤其是**影像分類**。

卷積神經網路

卷積神經網路在電腦視覺任務上比傳統的全連接神經網路有效

實作使用MNIST（ 28×28 灰階影像），模型由3個Conv2D層（2D卷積層）和2個MaxPooling2D層（最大池化層）堆疊而成。

```
[8] 0 秒
# 導入必要的函式庫
from tensorflow import keras
from tensorflow.keras import layers

# 定義模型的輸入層，輸入圖像的尺寸為28(圖片高度)x28(圖片寬度)像素，1(channel)代表灰度圖像(單通道)
inputs = keras.Input(shape=(28, 28, 1))

# 第一個卷積層：使用32個3x3大小的濾波器，激活函數為ReLU
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(inputs)

# 第一個最大池化層：將特徵圖的尺寸減半，池化窗口大小為2x2
x = layers.MaxPooling2D(pool_size=(2, 2))(x)

# 第二個卷積層：使用64個3x3大小的濾波器，激活函數為ReLU
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)

# 第二個最大池化層：將特徵圖的尺寸減半，池化窗口大小為2x2
x = layers.MaxPooling2D(pool_size=(2, 2))(x)

# 第三個卷積層：使用128個3x3大小的濾波器，激活函數為ReLU
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)

# 加入扁平層
# 將三維的特徵圖展平為一維向量，以便輸入到全連接層
x = layers.Flatten()(x)

# 輸出層：一個全連接層，有10個神經元(對應10個類別)，激活函數為Softmax用於多分類
outputs = layers.Dense(10, activation="softmax")(x)

# 建立Keras模型，指定輸入和輸出
model = keras.Model(inputs=inputs, outputs=outputs)
```

使用 Keras functional API 建模

測試準確率約 99%

```
[5] 2 秒
# 使用測試數據評估模型，返回測試損失 (test_loss) 和測試準確度 (test_acc)
test_loss, test_acc = model.evaluate(test_images, test_labels)

# 印出測試準確度
print(f"Test accuracy: {test_acc:.3f}")

... 313/313 ————— 3s 9ms/step - accuracy: 0.9898 - loss: 0.0359
Test accuracy: 0.992
```

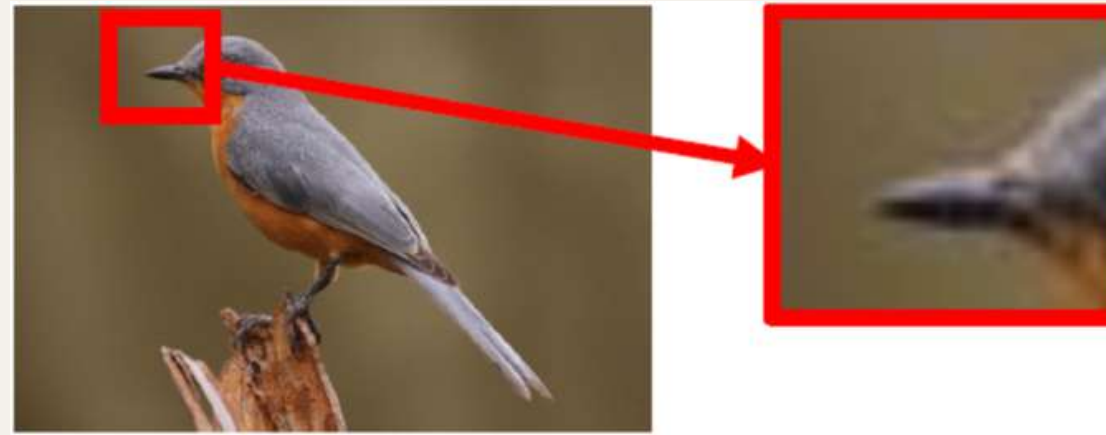
這邊只展示最初的建構模型與最後的模型測試，完整實作內容可查看：[實作文件檔](#)

卷積神經網路

為什麼簡單的卷積神經網路會有這麼好的表現/效果？

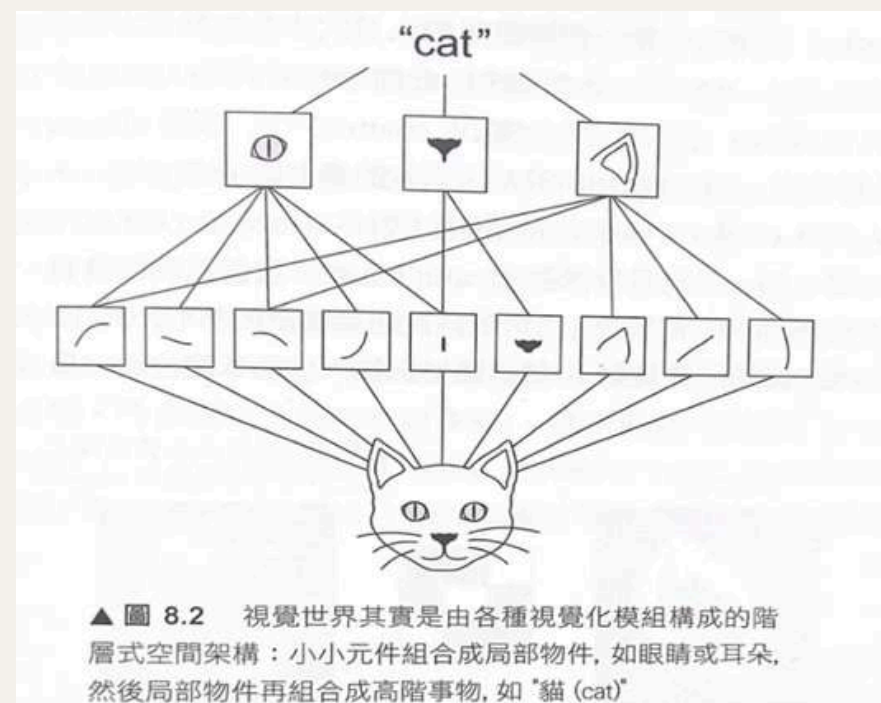
密集層：由輸入的特徵空間中學習全域的pattern

卷積層：學習局部的pattern



卷積層的兩種特性

1. **平移不變性**：影像在右下角學習到某個pattern，卷積神經網路可以在任何位置識別這個pattern。
2. **空間層次結構（由局部到整體）**：第一層卷積層，會學習局部圖案(小pattern)，第二層會基於第一層學習到的特徵來學習較大的pattern。



- 第一層卷積（低階特徵），學到：邊緣、線條、角落
- 第二層卷積（中階特徵），學到：形狀、部位
- 第三層卷積（高階特徵），學到：物件語意、整體結構

▲ 圖 8.2 視覺世界其實是由各種視覺化模組構成的階層式空間架構：小小元件組合成局部物件，如眼睛或耳朵，然後局部物件再組合成高階事物，如“貓 (cat)”

卷積神經網路

卷積層到底在做什麼？

卷積層會在三維特徵圖上，使用大小為 3×3 或 5×5 的卷積核進行滑動式運算。每一次運算都是針對一個三維區塊進行。當卷積核在特徵圖上滑動時，會擷取對應位置的區塊，並與卷積核權重進行逐元素相乘後加總，得到一個數值，代表該位置對應特徵的回應強度。

當卷積核完成整張特徵圖的掃描後，所有位置產生的數值會依原有空間排列方式組合，形成一張二維的輸出特徵圖。這些特徵圖再沿著深度方向堆疊，形成新的三維輸出特徵圖。

Padding

在卷積運算中，卷積核會以固定大小的視窗在特徵圖上由左至右、由上至下滑動。若不使用填補（padding），視窗必須完全落在輸入範圍內，導致邊界位置無法計算，輸出特徵圖的寬與高會小於輸入，此現象稱為邊界效應。為了解決此問題，可在輸入特徵圖四周補 0，使視窗能夠涵蓋原本的邊界位置。

步長（stride）

除了填補之外，步長（stride）也會影響輸出大小。步長代表卷積視窗每次滑動的距離，當 stride 增大時，可計算的位置減少，輸出特徵圖的解析度也隨之降低。

padding 用來保留邊界資訊，而 stride 則用來控制特徵圖的縮小程度

卷積神經網路

最大池化(Max-Pooling)操作

前面的範例中，每個MaxPooling2D層輸出的特徵圖長寬會減半。

例如：在第一個MaxPooling2D之前，特徵圖是 26×26 ，但在最大池化層操作後，空間軸的維度會減半成 13×13 。這就是最大池化層的作用：**積極對特徵圖進行降採樣，並輸出採樣區塊中的最大值。**

那我只用卷積，不用 Pooling 可不可以？

不太行，原因有兩個：

1. **不利於學習空間層次結構**：沒有 Pooling $\rightarrow$ 空間解析度一直很高
2. **參數量爆炸，導致過度配適**：
 - a. **最終特徵圖**： $22 \times 22 \times 128 \rightarrow \text{Flatten} = 61,952$
 - b. **Dense(10) $\approx$ 62 萬參數**

```
# 定義模型的輸入層，輸入圖像的尺寸為 28x28 像素，1 代表灰度圖像（單通道）
inputs = keras.Input(shape=(28, 28, 1))
# 第一個卷積層：使用 32 個 3x3 大小的濾波器，啟動函數為 ReLU
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(inputs)
# 第二個卷積層：使用 64 個 3x3 大小的濾波器，啟動函數為 ReLU
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
# 第三個卷積層：使用 128 個 3x3 大小的濾波器，啟動函數為 ReLU
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
# 將三維的特徵圖展平為一維向量，以便輸入到全連接層
x = layers.Flatten()(x)
# 輸出層：一個全連接層，有 10 個神經元（對應 10 個類別），啟動函數為 Softmax 用於多分類
outputs = layers.Dense(10, activation="softmax")(x)
# 建立 Keras 模型，指定輸入和輸出
model_no_max_pool = keras.Model(inputs=inputs, outputs=outputs)
```

model_no_max_pool.summary()

Model: "functional_4"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 28, 28, 1)	0
conv2d_12 (Conv2D)	(None, 26, 26, 32)	320
conv2d_13 (Conv2D)	(None, 24, 24, 64)	18,496
conv2d_14 (Conv2D)	(None, 22, 22, 128)	73,856
flatten_4 (Flatten)	(None, 61952)	0
dense_4 (Dense)	(None, 10)	619,530

Total params: 712,202 (2.72 MB)
Trainable params: 712,202 (2.72 MB)
Non-trainable params: 0 (0.00 B)

以少量資料集從頭訓練 一個卷積神經網路

實作任務與資料集設定

任務說明：

- 影像分類任務：貓 vs 狗
- 二元分類問題 → 使用 sigmoid + binary crossentropy

資料集概況

- 總共 5,000 張影像
 - 貓：2,500
 - 狗：2,500
- 資料切分方式：
 - 訓練集：2,000
 - 驗證集：1,000
 - 測試集：2,000

將資料集分成3個子資料集，架構如下：

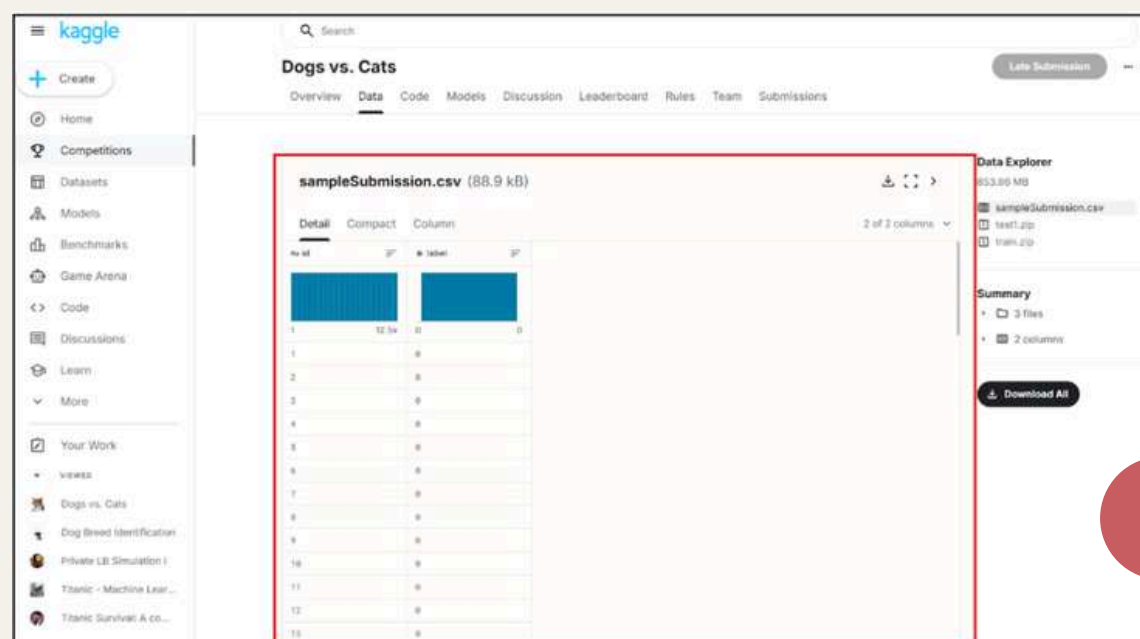
```
cats_vs_dogs_small/  
...train/ ← 訓練集  
.....cat/ ← 包含 1000 張貓的圖片  
.....dog/ ← 包含 1000 張狗的圖片  
...validation/ ← 驗證集  
.....cat/ ← 包含 500 張貓的圖片  
.....dog/ ← 包含 500 張狗的圖片  
...test/ ← 測試集  
.....cat/ ← 包含 1000 張貓的圖片  
.....dog/ ← 包含 1000 張狗的圖片
```

資料量其實不大！！！！

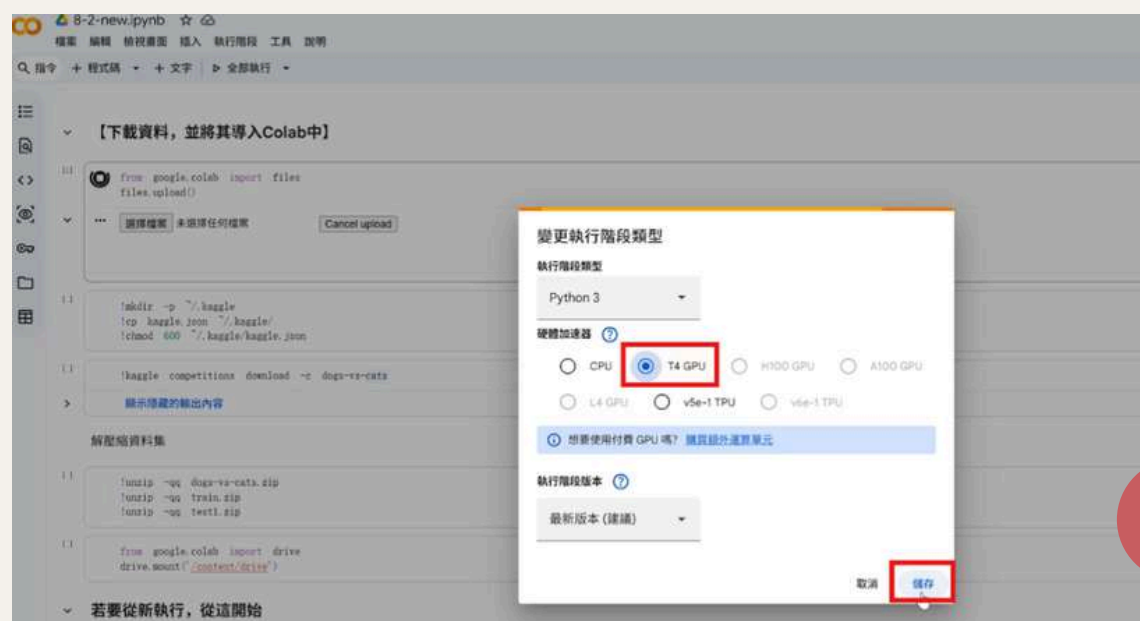
以少量資料集從頭訓練 一個卷積神經網路

資料取得與環境準備

- 資料集來源：Kaggle (Dogs vs Cats)
- 透過 Kaggle API 下載至 Colab
- 使用 **GPU** 作為訓練環境
- 將資料整理成：
 - train / validation / test
 - 並以資料夾結構對應類別



取得Kaggle資料集



使用GPU進行訓練

以少量資料集從頭訓練 一個卷積神經網路

建立神經網路

因為將處理更大的影像、更複雜的問題，因此需要擴大模型的規模，實際做法是組合更多的Conv2D（2D卷積層） + MaxPooling2D（最大池化層）。

```
[7] 10 秒  
✓  
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。  
from tensorflow.keras import layers # 從 tensorflow.keras 導入 layers 模組，用於構建神經網路層。  
  
inputs = keras.Input(shape=(180,180,3)) # 定義模型的輸入層，指定輸入圖片的形狀為 180x180 像素，3 個顏色通道（RGB）。  
  
x = layers.Rescaling(1./255)(inputs) # 將輸入圖片的像素值從 0-255 縮放到 0-1 範圍，進行數據正規化。  
x = layers.Conv2D(filters=32,kernel_size=3,activation='relu')(x) # 第一個卷積層：32 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。  
x = layers.MaxPooling2D(pool_size=2)(x) # 第一個最大池化層：池化窗口大小 2x2，用於下採樣。  
x = layers.Conv2D(filters=64,kernel_size=3,activation='relu')(x) # 第二個卷積層：64 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。  
x = layers.MaxPooling2D(pool_size=2)(x) # 第二個最大池化層。  
x = layers.Conv2D(filters=128,kernel_size=3,activation='relu')(x) # 第三個卷積層：128 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。  
x = layers.MaxPooling2D(pool_size=2)(x) # 第三個最大池化層。  
x = layers.Conv2D(filters=256,kernel_size=3,activation='relu')(x) # 第四個卷積層：256 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。  
x = layers.MaxPooling2D(pool_size=2)(x) # 第四個最大池化層。  
x = layers.Conv2D(filters=256,kernel_size=3,activation='relu')(x) # 第五個卷積層：256 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。  
x = layers.Flatten()(x) # 將卷積層的輸出展平為一維向量，以便連接到全連接層。  
outputs = layers.Dense(1,activation='sigmoid')(x) # 輸出層：一個神經元，使用 Sigmoid 啟動函數，用於二元分類（貓或狗）。  
model = keras.Model(inputs=inputs,outputs=outputs) # 創建 Keras 模型，指定輸入和輸出。
```

使用Rescaling層開始，主要是“調整整數輸入像素質的range”，從原始範圍0~255，調整至0~1。

```
[7] 10 秒  
✓  
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。  
from tensorflow.keras import layers # 從 tensorflow.keras 導入 layers 模組，用於構建神經網路層。  
  
inputs = keras.Input(shape=(180,180,3)) # 定義模型的輸入層，指定輸入圖片的形狀為 180x180 像素，3 個顏色通道（RGB）。  
  
x = layers.Rescaling(1./255)(inputs) # 將輸入圖片的像素值從 0-255 縮放到 0-1 範圍，進行數據正規化。  
x = layers.Conv2D(filters=32,kernel_size=3,activation='relu')(x) # 第一個卷積層：32 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。  
x = layers.MaxPooling2D(pool_size=2)(x) # 第一個最大池化層：池化窗口大小 2x2，用於下採樣。  
x = layers.Conv2D(filters=64,kernel_size=3,activation='relu')(x) # 第二個卷積層：64 個濾波器，卷積核大小 3x3，使用 ReLU 啟動函數。
```


以少量資料集從頭訓練 一個卷積神經網路

資料預處理流程

核心流程：

1. 讀取 JPEG 圖片
2. 解碼成 RGB 像素
3. 轉為浮點數張量
4. `resize` 成 180×180
5. 組成 batch (32 張)

這些不用自己處理！

這些步驟在Keras中，使用`image_dataset_from_directory()`函式可全部完整處理好！

```
[10] from tensorflow.keras.utils import image_dataset_from_directory # 從 tensorflow.keras.utils 導入 image_dataset_from_directory 函數，用於從目錄載入圖片資料集。

train_dataset = image_dataset_from_directory( # 載入訓練資料集。
    new_base_dir / 'train', # 指定訓練資料的目錄。
    image_size=(180,180), # 將圖片調整為 180x180 像素。
    batch_size=32 # 設定每個批次的圖片數量為 32。
)

validation_dataset = image_dataset_from_directory( # 載入驗證資料集。
    new_base_dir / 'validation', # 指定驗證資料的目錄。
    image_size=(180,180), # 將圖片調整為 180x180 像素。
    batch_size=32 # 設定每個批次的圖片數量為 32。
)

test_dataset = image_dataset_from_directory( # 載入測試資料集。
    new_base_dir / 'test', # 指定測試資料的目錄。
    image_size=(180,180), # 將圖片調整為 180x180 像素。
    batch_size=32 # 設定每個批次的圖片數量為 32。
)

... Found 2000 files belonging to 2 classes.
Found 1000 files belonging to 2 classes.
Found 2000 files belonging to 2 classes.
```


以少量資料集從頭訓練 一個卷積神經網路

初始訓練結果與問題診斷

訓練方式：

- 使用 validation data 監控模型表現
- 搭配 ModelCheckpoint：只保留驗證表現最好的模型

```
[12] 2分 鐘
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。

callback = [ # 定義回呼函數列表。
    keras.callbacks.ModelCheckpoint( # ModelCheckpoint 回呼函數用於在訓練過程中儲存模型。
        filepath='convnet_from_scratch.keras', # 指定儲存模型的路徑和檔案名。
        save_best_only=True, # 只儲存驗證損失最小（最佳）的模型。
        monitor='val_loss')] # 監控驗證損失（validation loss）來判斷模型好壞。

history = model.fit( # 訓練模型。
    train_dataset, # 指定訓練資料集。
    epochs=30, # 設定訓練週期為 30。
    validation_data=validation_dataset, # 指定驗證資料集，用於在每個週期結束後評估模型性能。
    callbacks=callback # 傳入回呼函數列表。
)
```

... Epoch 1/30
63/63 ————— 15s 137ms/step - accuracy: 0.5161 - loss: 0.7072 - val_accuracy: 0.5000 -
Epoch 2/30

結果觀察

測試準確度是71%，由於訓練樣本較少，過度配適成為訓練時的首要顧慮因素。

```
[22] 4 秒
test_model = keras.models.load_model('convnet_from_scratch.keras') # 載入之前訓練的模型。

test_loss, test_acc = test_model.evaluate(test_dataset) # 在測試資料集上評估模型。
print(f'Test accuracy: {test_acc:.3f}') # 印出測試準確度。
```

63/63 ----- 4s 50ms/step - accuracy: 0.6942 - loss: 0.5930
Test accuracy: 0.710

還有一個方法可以避免過度配適：資料擴增法.....

以少量資料集從頭訓練一個卷積神經網路

解法一：資料擴增

在不增加新資料的前提下，透過隨機變換產生「不同樣貌的訓練樣本」，以增加資料多樣性，降低模型記憶特定樣本的風險。

使用的擴增方式

實作上，只要在模型的一開始加入資料擴增層就可以達到“資料擴增”這個目標。

定義擴增層：

```
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。
from tensorflow.keras import layers # 從 tensorflow.keras 導入 layers 模組。

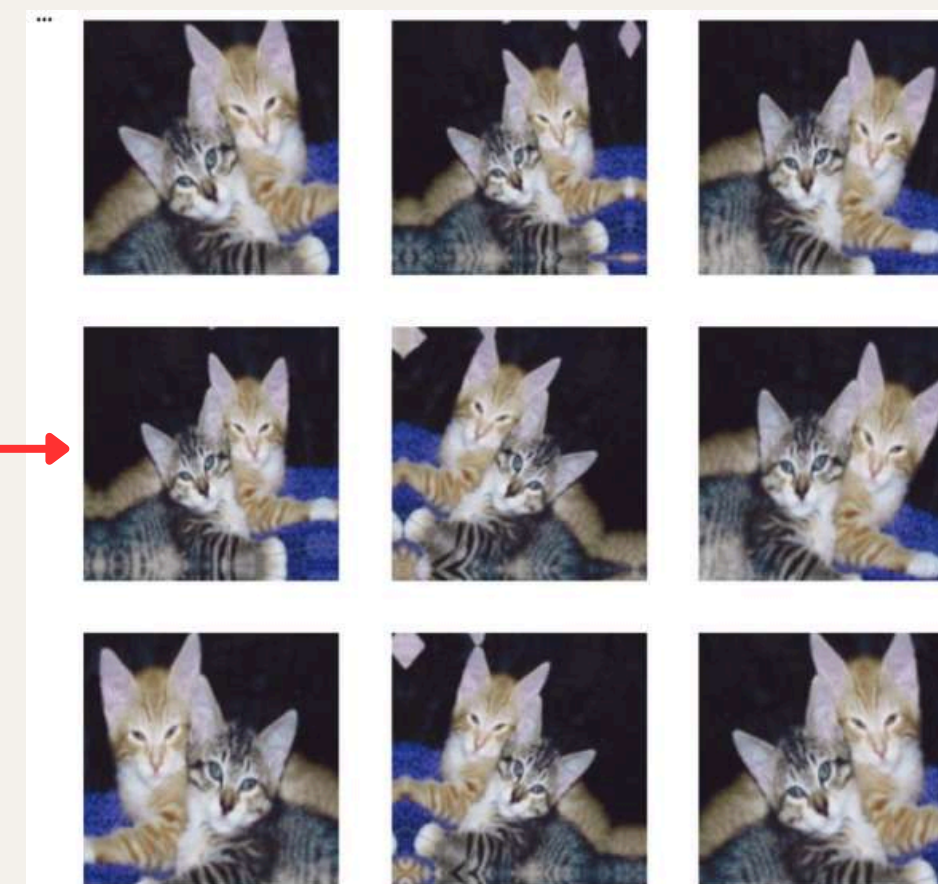
data_augmentation = keras.Sequential( # 定義資料擴增層的序列。
    [
        layers.RandomFlip('horizontal'), # 水平翻轉圖片。
        layers.RandomRotation(0.1), # 隨機旋轉圖片，角度範圍為 +/- 0.1 弧度。
        layers.RandomZoom(0.2), # 隨機縮放圖片，縮放因子範圍為 +/- 0.2。
    ]
)
```

- RandomFlip（水平翻轉）
- RandomRotation（小角度旋轉）
- RandomZoom（隨機縮放）

檢視經過“資料擴增”後的影像

```
[30]
✓ 1秒
import matplotlib.pyplot as plt # 導入 Matplotlib 庫的 pyplot 模組。

plt.figure(figsize=(10,10)) # 創建一個新的圖表，並設定圖表大小。
for images, _ in train_dataset.take(1): # 從訓練資料集中取出一個批次的圖片。
    for i in range(9): # 遍歷前 9 張圖片。
        augmented_images = data_augmentation(images) # 對圖片執行數據增強。
        ax = plt.subplot(3,3,i+1) # 在 3x3 的網格中創建子圖。
        plt.imshow(augmented_images[0].numpy().astype('uint8')) # 顯示增強後的圖片。
        plt.axis('off') # 關閉座標軸。
plt.show() # 顯示圖片。
```



以少量資料集從頭訓練 一個卷積神經網路

進階防過擬合：Dropout

因擴增影像仍高度相關，模型仍可能記住模式。因此在分類器前加入 Dropout，隨機關閉部分神經元，降低神經元間的共適應

```
[20] 0 秒
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。
from tensorflow.keras import layers # 從 tensorflow.keras 導入 layers 模組。

input_layer = keras.Input(shape=(180,180,3)) # 定義模型的輸入層，指定輸入圖片的形狀。
x = data_augmentation(input_layer) # 將輸入圖片傳遞給數據增強層。
x = layers.Rescaling(1./255)(x) # 將像素值縮放到 0-1 範圍。
x = layers.Conv2D(filters=32, kernel_size=3, activation='relu')(x) # 第一個卷積層，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第一個最大池化層。
x = layers.Conv2D(filters=64, kernel_size=3, activation='relu')(x) # 第二個卷積層，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第二個最大池化層。
x = layers.Conv2D(filters=128, kernel_size=3, activation='relu')(x) # 第三個卷積層，使用 ReLU 啟動函數。
x = layers.MaxPooling2D(pool_size=2)(x) # 第三個最大池化層。
x = layers.Conv2D(filters=256, kernel_size=3, activation='relu')(x) # 第四個卷積層，使用 ReLU 啟動函數。
x = layers.Flatten()(x) # 展平操作。
x = layers.Dropout(0.5)(x) # Dropout 層，以 0.5 的機率隨機關閉神經元，防止過擬合。
outputs = layers.Dense(1, activation='sigmoid')(x) # 輸出層，一個神經元，使用 Sigmoid 啟動函數。
model = keras.Model(inputs=input_layer, outputs=outputs) # 創建 Keras 模型。

model.compile(loss='binary_crossentropy', # 配置模型，損失函數為二元交叉熵。
              optimizer='rmsprop', # 優化器為 RMSprop。
              metrics=['accuracy']) # 評估指標為準確度。
```

改良後結果與比較

測試準確率提升至 83.3%

```
[36] 2 秒
from tensorflow import keras # 從 TensorFlow 導入 Keras 模組。
import os # 導入 os 模組。

model_filepath = 'convnet_from_scratch_with_augmentation.keras' # 定義模型檔案路徑。

if os.path.exists(model_filepath): # 檢查模型檔案是否存在。
    test_model = keras.models.load_model(model_filepath) # 載入最佳模型。
    test_loss, test_acc = test_model.evaluate(test_dataset) # 在測試資料集上評估模型。
    print(f'Test accuracy: {test_acc:.3f}') # 印出測試準確度。
else: # 如果檔案不存在。
    print(f'Error: Model file {model_filepath} not found.') # 印出錯誤訊息。
    print("Please ensure the training cell (cell 5k7lmQoYfZ7e) was executed successfully to save the model.") # 提醒用戶。

... 63/63 ----- 2s 30ms/step - accuracy: 0.8351 - loss: 0.4590
Test accuracy: 0.833
```

過度配適情況明顯改善，但從零訓練 CNN 在小資料集上仍有限制，很難再
向上突破..... → 需要引入 預訓練模型

使用預先訓練好的模型

使用預訓練模型的兩種策略

- 定義：事先以大量資料集訓練過的優秀模型
- 核心優勢：站在巨人的肩膀上，利用已學習的特徵知識

ImageNet資料集介紹

- 規模：140萬個標註影像
- 多樣性：1000個不同類別
- 內容：包含多種動物類別，如不同品種的貓狗等
- 地位：深度學習領域的重要基準（benchmark）資料集

VGG16神經網路

- 來源：已在Keras中預先安裝
- 訓練基礎：使用ImageNet訓練完成
- 架構特色：深層卷積神經網路，具有強大的特徵提取能力

```
[21]  
✓ 0 秒  
  
import keras  
# 這一行導入了 Keras 深度學習函式庫，讓我們可以使用其中的功能，例如預訓練模型。  
  
conv_base = keras.applications.vgg16.VGG16(  
    # 這一行初始化了一個 VGG16 卷積神經網路模型。VGG16 是一個在 ImageNet 資料集上預訓練過的經典模型。  
    weights='imagenet',  
    # 這個參數指定了模型要加載的權重。'imagenet' 表示使用在 ImageNet 資料集上訓練好的權重，這是一個大型的圖像分類資料集。  
    include_top=False,  
    # 這個參數設定為 False 意味著我們不包括模型的頂部（即分類層）。當我們將預訓練模型作為特徵提取器用於新的分類任務時，通常會這麼做，因為我們需要添加自定義的分類器。  
    input_shape=(180, 180, 3))  
# 這個參數定義了模型輸入圖像的形狀。這裡設定為 (180, 180, 3) 表示輸入圖像的高度為 180 像素，寬度為 180 像素，3 表示圖像有三個顏色通道（紅、綠、藍）。  
  
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16\_weights\_tf\_dim\_ordering\_tf\_kernels\_notop.h5  
58889256/58889256 ————— 0s 0us/step
```


使用預先訓練好的模型

策略一：特徵萃取 (Feature Extraction)

- 做法：將預訓練模型當作「特徵提取器」
- 原理：凍結整個卷積基底，只訓練自定義的分類層

沒有資料擴增的快速特徵萃取

1. 使用 `preprocess_input()` 預處理圖像
2. 呼叫 `conv_base.predict()` 萃取特徵（輸出為 NumPy 陣列）
3. 在萃取的特徵上訓練密集分類層

搭配資料擴增的特徵萃取

- 關鍵技術：凍結 (Freezing)
- 凍結定義：在訓練期間禁止更新權重
- 實作方法：設定 `trainable=False`

```
[29] ✓ con_base = keras.applications.vgg16.VGG16(  
      weights='imagenet',  
      include_top=False)  
      con_base.trainable = False
```

最終成果

- 測試準確度：97.9%
- 效果提升：過度配適情況明顯改善

```
[59] ✓ 6 秒 test_model = keras.models.load_model(  
      "feature_extraction_with_data_augmentation.keras")  
      test_loss, test_acc = test_model.evaluate(test_dataset)  
      print(f"Test accuracy: {test_acc:.3f}")  
      ... 63/63 ----- 6s 91ms/step - accuracy: 0.9740 - loss: 2.5900  
      Test accuracy: 0.979
```


使用預先訓練好的模型

策略二：微調（Fine-tuning）

- 與特徵萃取的差異：不是凍結整個卷積基底，而是解凍「頂部」某些層
- 目的：讓預訓練模型適應新的特定任務

實作細節

1. 凍結：block4_pool之前的所有層
2. 解凍：block5_conv1、block5_conv2、block5_conv3

訓練技巧

- 優化器：RMSProp
- 學習率：非常低
- 原因：避免損害預訓練權重的表示法

最終成果

- 測試準確度：98%
- 顯著提升：相較於從零訓練或僅特徵萃取

```
[38] 6 秒
✓
model = keras.models.load_model("fine_tuning.keras")
test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")

63/63 ----- 6s 92ms/step - accuracy: 0.9765 - loss: 1.9686
Test accuracy: 0.980
```